

Modelos Generativos Profundos

Clase 13: Redes generativas adversarias (parte II)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Clase de hoy

Recuerdo clase anterior

Convolución transpuesta

DCGAN

SAGAN

ProGAN

StyleGAN

U-Net

Recuerdo clase anterior

Idea general

Una GAN es un modelo de variable latente, $p_\theta(x, z) = p(z)p_\theta(x|z)$ con $p(z) = \mathcal{N}(0, \mathbf{I}_L)$ y $p_\theta(x|z) = \delta_{G_\theta(z)}(x)$, donde $G_\theta : \mathbb{R}^L \rightarrow \mathbb{R}^D$ es una red neuronal.

La red generadora G_θ es entrenada para engañar a un discriminador $q_\phi(y|x) \sim \text{Bernoulli}(D_\phi(x))$, con $D_\phi : \mathbb{R}^D \rightarrow [0, 1]$ una red neuronal, el cual a su vez aprende a diferenciar muestras provenientes de $p_{\text{data}}(x)$ ($y = 1$) o de $p_\theta(x)$ ($y = 0$).

Función objetivo

Si el clasificador $q_\phi(y|x)$ es optimizado mediante máxima verosimilitud, entonces una GAN se reduce al siguiente juego:

$$\min_{\theta} \max_{\phi} \mathbb{E}_{p_{\text{data}}(x)} [\log D_{\phi}(x)] + \mathbb{E}_{z \sim \mathcal{N}(0, I_L)} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

En la práctica, se suele utilizar una versión no saturante en el generador, llegando a las siguientes funciones de pérdida:

$$\mathcal{L}_D(\phi) = -\mathbb{E}_{p_{\text{data}}(x)} [\log D_{\phi}(x)] - \mathbb{E}_{z \sim \mathcal{N}(0, I_L)} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

$$\mathcal{L}_G(\theta) = -\mathbb{E}_{z \sim \mathcal{N}(0, I_L)} [\log D_{\phi}(G_{\theta}(z))] .$$

Convolución transpuesta

Las redes convolucionales permiten inducir un sesgo de **localidad** e **invarianza a traslaciones** de features dentro de la entrada, lo que las vuelve útiles para el diseño de arquitecturas neuronales para imágenes.

Por otro lado, dada la dificultad de entrenar una GAN, es necesario utilizar arquitecturas específicas que permitan estabilizar el entrenamiento y obtener buenos resultados.

Si bien muchas de estas heurísticas son solo empíricas, varias de ellas siguen siendo importantes en la actualidad.

Motivación CNN transpuestas

Por otro lado, una convolución estándar suele reducir la dimensión de entrada de la imagen (o en el mejor de los casos, mantenerla).

En una GAN se suele comenzar con una representación latente de baja dimensión, la cual va aumentando progresivamente a medida que se avanza en la red neuronal.

La convolución transpuesta es una operación dual que permite aumentar la dimensión de la entrada, lo que la hace útil para el diseño de arquitecturas de generadores en GANs.

Convolución como operador lineal

Dada una entrada $x \in \mathbb{R}^{D_{\text{in}}}$, una **convolución** con **filtro** $w \in \mathbb{R}^K$, **stride** $S \geq 1$ y **padding** $P \geq 0$ produce una salida $y \in \mathbb{R}^{D_{\text{out}}}$ con

$$y_d = \sum_{k=1}^K w_k x_{S(d-1)+k-P}, \quad d = 1, \dots, D_{\text{out}},$$

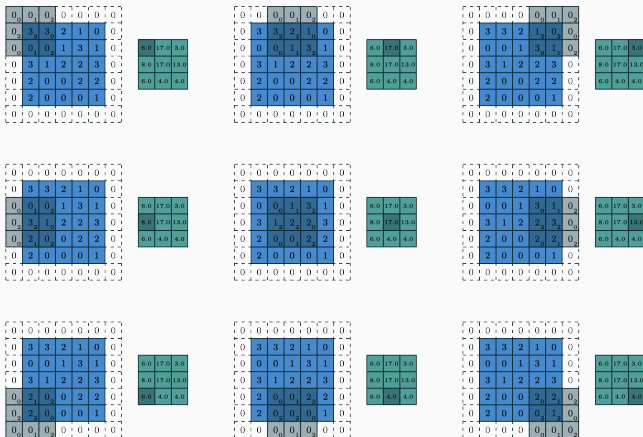
donde $x_d = 0$ para $d \notin \{1, \dots, D_{\text{in}}\}$ (zero-padding implícito) y

$$D_{\text{out}} = \left\lfloor \frac{D_{\text{in}} + 2P - K}{S} \right\rfloor + 1.$$

Notar que esta operación se puede escribir como $y = Ax$, donde $A \in \mathcal{M}_{D_{\text{out}}, D_{\text{in}}}(\mathbb{R})$ es una **matriz de Toeplitz**. En particular, la convolución es una transformación lineal.

Convolución como operador lineal

La siguiente imagen muestra una convolución 2D con hiperparámetros $K = 3$, $P = 1$ y $S = 2$:



Convolución transpuesta

Dada una convolución con matriz $A \in \mathcal{M}_{D_{\text{out}}, D_{\text{in}}}(\mathbb{R})$, la **convolución transpuesta** es el operador $A^T : \mathbb{R}^{D_{\text{out}}} \rightarrow \mathbb{R}^{D_{\text{in}}}$:

- En particular, si $D_{\text{out}} < D_{\text{in}}$ (como en una convolución estándar), la convolución transpuesta aumenta la dimensión de la entrada.
- Más precisamente, el tamaño de la salida de una convolución transpuesta con entrada $y \in \mathbb{R}^{D_{\text{out}}}$ es:

$$D_{\text{in}} = S(D_{\text{out}} - 1) + K - 2P.$$

En particular, el caso estándar $K = 4$, $S = 2$, $P = 1$ duplica la dimensión de salida ($D_{\text{in}} = 2D_{\text{out}}$).

Convolución transpuesta

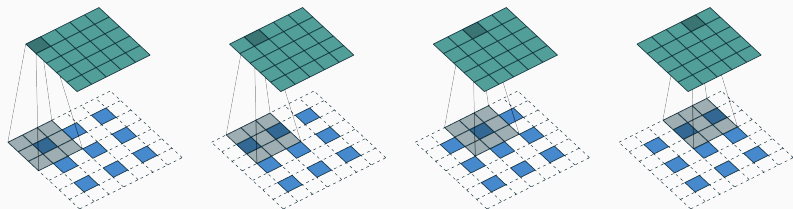
En la práctica, la convolución transpuesta es equivalente a intercalar $S-1$ ceros entre los elementos de la entrada y aplicar una convolución estándar con stride 1.

Esto motiva a llamar a la convolución transpuesta convolución con **stride fraccionario**.

Por otro lado, la convolución y la convolución transpuesta 1D se puede extender a varias dimensiones aplicándola por separado en cada dimensión, con sus propios hiperparámetros K , S y P .

Convolución transpuesta

La siguiente imagen muestra una convolución transpuesta 2D con hiperparámetros $K = 3$, $P = 1$ y $S = 2$:



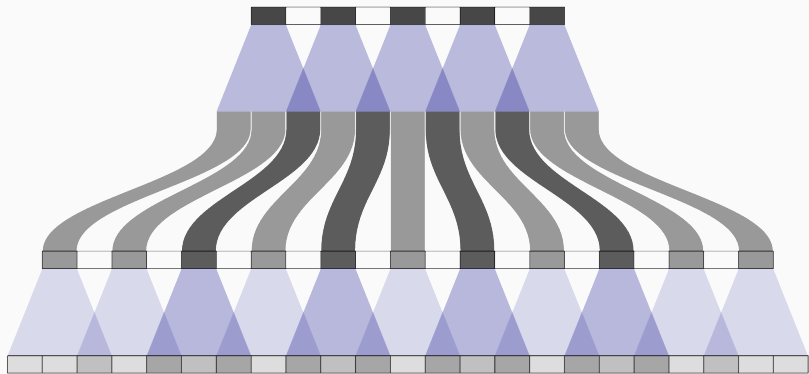
Checkerboard artifact

En una convolución transpuesta, si S no es múltiplo de K , cada posición de la salida recibirá un número desigual de contribuciones del filtro, produciendo un patrón periódico en la imagen generada.

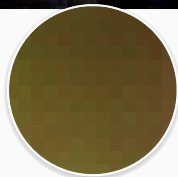
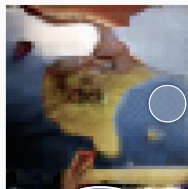
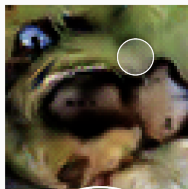
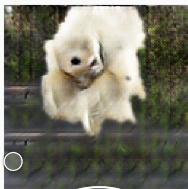
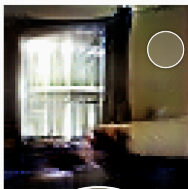
Existen dos estrategias para evitar este fenómeno:

1. Elegir $K = 2S$, de modo que todas las posiciones reciban el mismo número de contribuciones.
2. Realizar interpolación clásica (e.g. upsampling bilineal o nearest-neighbor) seguido de una convolución estándar.

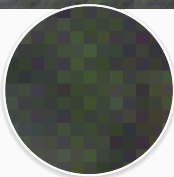
Checkerboard artifact



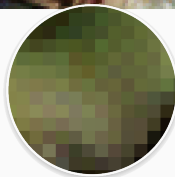
Checkerboard artifact



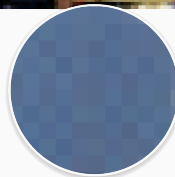
Radford, et al., 2015 [1]



Salimans et al., 2016 [2]



Donahue, et al., 2016 [3]



Dumoulin, et al., 2016 [4]

DCGAN

Deep convolutional GAN

DCGAN fue uno de los primeros trabajos en obtener buenos resultados a resolución 64×64 .

Este trabajo estableció empíricamente algunos principios básicos para el diseño de arquitecturas de GANs para imágenes:

1. Usar arquitecturas **completamente convolucionales**.
2. **Batch normalization** en todas las capas, excepto en la salida de G_θ y la entrada de D_ϕ .
3. Convoluciones con stride en lugar de pooling.
4. Activaciones ReLU en G_θ y LeakyReLU en D_ϕ .
5. Activación **tanh** en la salida de G_θ .
6. Durante la optimización, usar Adam con $\beta_1 = 0.5$.

Propiedades del espacio latente

Un resultado relevante de DCGAN fue verificar que el espacio latente de una GAN permite realizar **interpolación semántica**.

Dadas dos muestras latentes, $z^0, z^1 \sim \mathcal{N}(0, I_L)$, la curva

$$t \mapsto G_{\theta}((1-t)z_0 + tz_1), \quad t \in [0, 1],$$

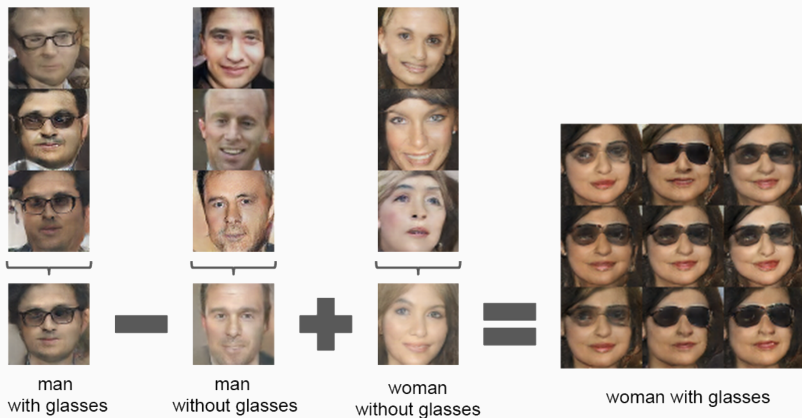
produce transiciones visuales suaves entre los atributos de las muestras $G_{\theta}(z_0)$ y $G_{\theta}(z_1)$.

Más aún, DCGAN mostró que es posible realizar **aritmética vectorial** el espacio latente, pudiendo modificar atributos específicos de la imagen generada sumando o restando vectores latentes directores correspondientes a esos atributos.

Propiedades del espacio latente



Propiedades del espacio latente



SAGAN

Self-attention GAN

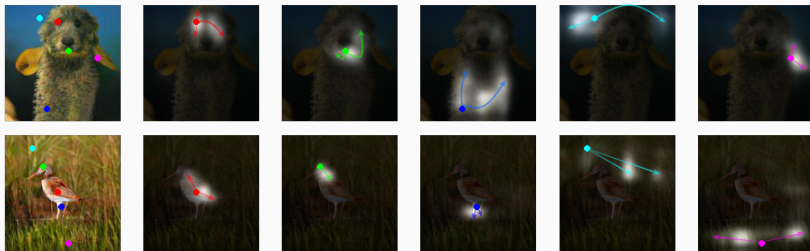
Las CNNs suelen presentar dificultades para modelar dependencias de largo alcance debido a su naturaleza local.

SAGAN propone incorporar considerar cada pixel $(x_1, \dots, x_N) \subset \mathbb{R}^C$ (con $N = H \times W$) como un token y aplicar un módulo de self-attention que permita capturar dependencias entre cualquier par de posiciones del feature map con un único bloque:

$$y_i = \gamma \left(W_O \sum_{j=1}^N a_{ij} (W_V x_j) \right) + x_i, \quad \text{donde} \quad a_{ij} = \frac{\exp(\langle W_Q x_i, W_K x_j \rangle)}{\sum_t \exp(\langle W_Q x_i, W_K x_t \rangle)}$$

El parámetro entrenable $\gamma \in \mathbb{R}$ se inicializa en 0 para que la red comience como una CNN pura e incorpore dependencias de largo alcance gradualmente durante la optimización.

Mapa de atención



Hinge adversarial loss

SAGAN también utiliza otras heurísticas de diseño como **normalización espectral** o **TTUR**, las cuales permiten ver la dificultad de entrenar GANs.

Otro cambio usual en GANs es variar otra función de pérdida. En el caso de SAGAN, la **hinge adversarial loss** trata a D_ϕ como un margen de SVM:

$$\mathcal{L}_D(\phi) = -\mathbb{E}[\min(0, -1 + D_\phi(x))] - \mathbb{E}[\min(0, -1 - D_\phi(G_\theta(z)))]$$

$$\mathcal{L}_G(\theta) = -\mathbb{E}[D_\phi(G_\theta(z))],$$

donde ahora $D_\phi(x) \in \mathbb{R}$ es un score de clasificación.

ProGAN

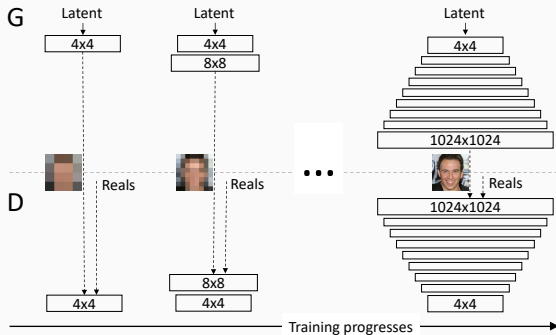
ProGAN fue el primer método en producir imágenes convincentes a una resolución de 1024×1024 .

En este enfoque, G_θ y D_ϕ comienzan en una resolución de 4×4 y van agregando progresivamente pares de capas hasta la resolución objetivo (**curriculum learning**).

Al igual que los trabajos anteriores, ProGAN también propone otro conjunto de heurísticas que pueden ayudar a estabilizar el entrenamiento.

Entre ellas, está usar la pérdida de **Wasserstein GAN**, la cual relaciona directamente las GANs con transporte óptimo.

Arquitectura



Para poder ampliar la resolución apilando más capas convolucionales, ProGAN utiliza dos convoluciones 1×1 necesarias durante el entrenamiento:

- $\text{toRGB}_r : \mathbb{R}^{C \times r \times r} \rightarrow \mathbb{R}^{3 \times r \times r}$ (usada por el generador).
- $\text{fromRGB}_r : \mathbb{R}^{3 \times r \times r} \rightarrow \mathbb{R}^{C \times r \times r}$ (usada por el discriminador).

Con esto, durante la transición $r \rightarrow 2r$, para $\alpha \in [0, 1]$ que aumenta linealmente durante el entrenamiento, se considera

- $G_{\text{out}} = (1 - \alpha) \text{toRGB}_{2r}(\text{up}(h_r)) + \alpha \text{toRGB}_{2r}(\text{conv}(\text{up}(h_r)))$.
- $D_{\text{in}} = (1 - \alpha) \text{fromRGB}_r(\text{down}(x_{2r})) + \alpha \text{fromRGB}_{2r}(\text{conv}(\text{down}(u_{2x})))$.

donde h_r es el feature map de G_θ en resolución $r \times r$ y x_{2r} la imagen real en resolución $2r \times 2r$.

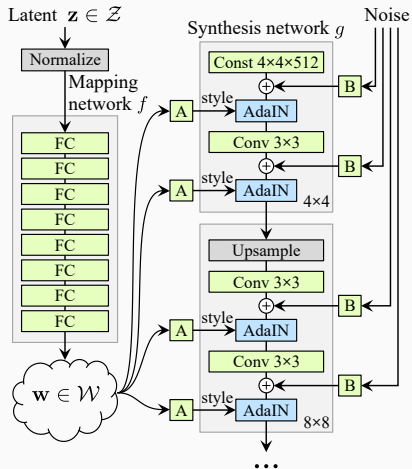
StyleGAN

Si bien ProGAN permite generar imágenes de alta calidad, no permite controlar fácilmente atributos específicos de la imagen generada, lo que limita su aplicabilidad.

StyleGAN es una arquitectura de generador que permite desacoplar la distribución del espacio latente de la GAN para poder realizar un mejor control de los atributos de la imagen generada.

Para esto, inyecta información de estilo en cada capa de la red mediante métodos de **modulación**.

Arquitectura



Mapping network

Para empezar, StyleGAN mapea la representación latente $z \in \mathbb{R}^L$ a una representación intermedia $w \in \mathbb{R}^L$ mediante una **mapping network** $f_\psi : \mathbb{R}^L \rightarrow \mathbb{R}^L$.

La idea es que f_ψ pueda aprender a transformar la distribución $p_\theta(z) \sim \mathcal{N}(0, I_L)$ a una distribución más adecuada para el control de atributos en w .

Esto es necesario ya que, sin esta red, G_θ debe doblarse para evitar generar muestras poco probables, lo que rompe la linealidad semántica del espacio latente.

Synthesis network

La **synthesis network** $g_w : \mathbb{R}^L \rightarrow \mathbb{R}^{3 \times H \times W}$ genera imágenes inyectando la representación intermedia w en cada capa de la red utilizando **adaptative instance normalization (AdaIN)**:

$$\text{AdaIN}(x_c, w) = \alpha_c \frac{x_c - \mu(x_c)}{\sigma(x_c)} + \beta_c \in \mathbb{R}^{H \times W},$$

donde $x_c \in \mathbb{R}^{H \times W}$ es el c -ésimo canal de la capa, $\mu(x_c)$ y $\sigma(x_c)$ son su media y desviación estándar, y $(\alpha, \beta) \in \mathbb{R}^C \times \mathbb{R}^C$ provienen de una transformación afín aprendida, $A : \mathbb{R}^L \rightarrow \mathbb{R}^C \times \mathbb{R}^C$.

De esta forma, la red comienza desde un tensor inicial $c_0 \in \mathbb{R}^{C_0 \times H_0 \times W_0}$ y luego cada capa aplica AdaIN con un estilo específico definido por w .

La técnica anterior permite controlar atributos específicos de la imagen generada, ya que cada capa de la red controla atributos de diferente nivel de abstracción.

StyleGAN también realiza **style mixing**, donde se cruzan dos latentes w_1 , w_2 en una capa ℓ^* aleatoria. En estos casos, capas de baja resolución controlan pose y estructura, mientras que capas de alta resolución controlan textura y detalles finos.

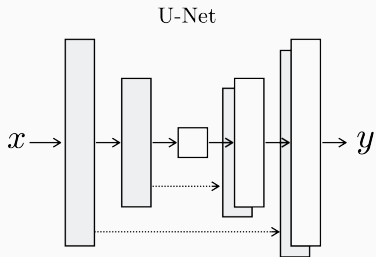
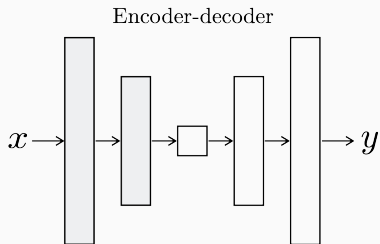
U-Net

Arquitectura U-Net

Cuando la condición de una CGAN condicional es una imagen, es usual requerir una arquitectura que preserve la estructura espacial. La **U-Net** es un autoencoder completamente convolucional con **skip connections** entre encoder y decoder:

- El **encoder** reduce progresivamente la resolución extrayendo representaciones de alto nivel.
- El **decoder** recupera la resolución mediante upsampling y concatena, en cada nivel, la activación homóloga del encoder.
- Estas skip connections permiten transmitir directamente información espacial de bajo nivel desde el encoder hacia el decoder, sin pasar por el cuello de botella.

Arquitectura U-Net



En un comienzo, la U-Net fue propuesta para **segmentación semántica** (clasificación por píxel), pero su arquitectura se volvió muy popular para tareas de generación de imágenes:

- En GANs condicionales image-to-image (e.g., **pix2pix**), el generador suele ser una U-Net.
- En modelos de difusión, la U-Net modificada con mecanismos de atención permite generar imágenes de alta calidad y condicionar con respecto a texto.

En la próxima clase:

- Transferencia de estilo.
- Nociones de transporte óptimo.
- Algunas cosas de las GANs.

Modelos Generativos Profundos

Clase 13: Redes generativas adversarias (parte II)